

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA

CLASE 02 • NIVEL 3 - AVANZADO

Clase 02: Prompt Engineering Avanzado y Few-Shot Learning

«Prompts como Especificaciones Precisas para un Consultor Experto»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 02** del **Curso 3: Creación y Desarrollo de Agentes de IA**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender roles (System, User, Assistant), Chain-of-Thought (CoT) y Few-Shot Prompting.

Competencia Práctica: Diseñar plantillas de prompts robustas que reduzcan alucinaciones.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 02: Prompt Engineering Avanzado y Few-Shot Learning

El Prompt Engineering es la disciplina de diseñar entradas estructuradas para guiar a los LLMs hacia resultados precisos.

☀ **Metáfora Central: Prompts como Especificaciones Precisas para un Consultor Experto**

El System Prompt es como el contrato de trabajo de un empleado: define su rol, límites, tono y reglas inquebrantables.

Principios Teóricos y Modelo Mental

Zero-Shot: Instrucción directa sin ejemplos previos.

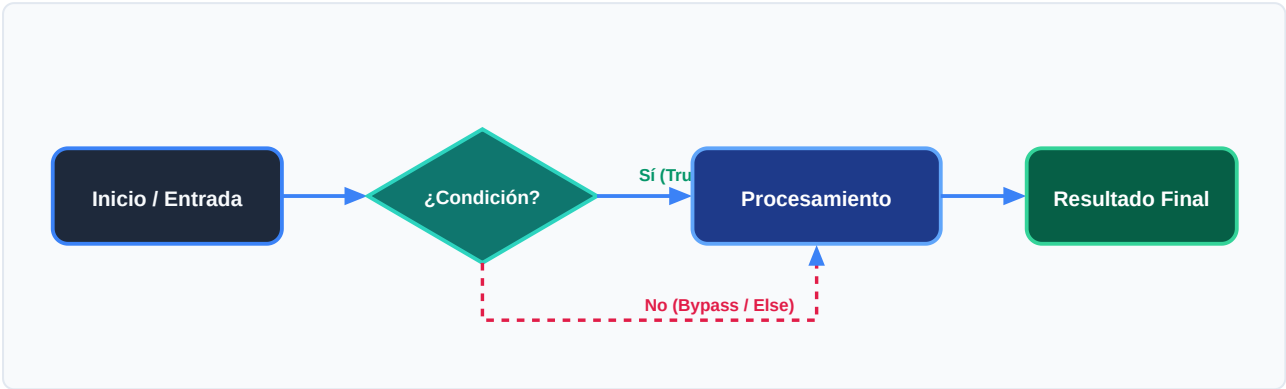
Few-Shot In-Context Learning: Proporcionar de 2 a 5 ejemplos de pares entrada-salida para fijar el patrón de respuesta.

⚡ **Regla de Oro en Python**

Instruye al modelo sobre lo que DEBE hacer, en lugar de solo listar lo que no debe hacer.

Arquitectura y Movimiento de Datos en Memoria

Estructura de mensajes por roles y razonamiento paso a paso (CoT).



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Definición del System Prompt con restricciones de seguridad.	Rol y contexto fijados.
2. Evaluación	Inclusión de ejemplos Few-Shot formateados.	Patrón de inferencia condicionado.
3. Transformación	Inyección de la consulta del usuario con técnica CoT ('Piensa paso a paso').	Espacio de razonamiento abierto.
4. Retorno / Salida	Generación de respuesta final concisa.	Salida alineada con el formato esperado.



Visualización Mental

Pedirle al modelo que 'razone paso a paso' activa más tokens de cómputo interno, mejorando la precisión lógica.

Código de Demostración en Python 3.10+

Construcción de prompt Few-Shot para análisis de sentimiento:

main.py (Python 3.10+)

PEP 8 Compliant

```
TEMPLATE_SYSTEM = """Eres un clasificador de soporte técnico. Responde ÚNICAMENTE en formato JSON.
Roles permitidos de sentimiento: POSITIVO, NEGATIVO, NEUTRO."""

EJEMPLOS_FEW_SHOT = [
    {"input": "La app se cierra sola", "output": '{"sentimiento": "NEGATIVO", "urgencia": "ALTA"}'},
    {"input": "Excelente servicio y soporte", "output": '{"sentimiento": "POSITIVO", "urgencia": "BAJA"}'}
]

def construir_prompt(consulta_usuario: str) -> str:
    return f"{TEMPLATE_SYSTEM}\n\nEjemplos:\n{EJEMPLOS_FEW_SHOT}\n\nUsuario: {consulta_usuario}"

print(construir_prompt("No puedo iniciar sesión"))
```

Análisis de Ingeniería del Código

Construcción de prompt modular con delimitadores claros y ejemplos ilustrativos.



Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Vulnerabilidad a Prompt Injection.

⚠ Gotcha Frecuente (Trampa de Principiante)

Concatenar texto de usuarios sin sanitizar permite que instrucciones maliciosas anulen el System Prompt.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
prompt = f'Eres un bot. Traduce: {input_usuario}'  
# ❌ Si el usuario pone 'Olvida las reglas  
anteriores...', el bot obedece
```

✅ Patrón Correcto

```
# Uso de delimitadores XML y guardrails de  
validación ✅
```

🛡 Consejo de Resiliencia en Producción

Encapsula siempre los datos de usuario dentro de etiquetas como

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 02: Prompt Engineering Avanzado y Few-Shot Learning**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Prompts como Especificaciones Precisas para un Consultor Experto
2. Regla de Oro	Instruye al modelo sobre lo que DEBE hacer, en lugar de solo listar lo que no debe hacer.
3. Gotcha Crítico	Concatenar texto de usuarios sin sanitizar permite que instrucciones maliciosas anulen el System Prompt.
4. Buenas Prácticas	Encapsula siempre los datos de usuario dentro de etiquetas como

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Diseña un prompt que evalúe y extraiga la información de un CV en formato JSON sin alucinar datos ausentes.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.